



give examples context in github actions workflow yml and explain

Here are clear GitHub Actions context examples in workflow YAML, with explanations for each one. [\[1\]](#) [\[2\]](#)

What context means

A context is an object GitHub Actions makes available during a workflow run, and you access it with expression syntax like `${{ github.ref }}`. [\[1\]](#)

These values help your workflow react to the event, branch, job status, runner OS, matrix values, and secrets. [\[3\]](#) [\[1\]](#)

Example workflow

```
name: CI

on:
  push:
    branches:
      - main
      - develop
  pull_request:

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Show GitHub context
        run: echo "Repo is ${ github.repository }, ref is ${ github.ref }"

      - name: Use env context
        env:
          APP_ENV: production
        run: echo "Environment is ${ env.APP_ENV }"

      - name: Use runner context
        run: echo "Runner OS is ${ runner.os }"

      - name: Use secrets context
        run: echo "Token length is ${#${ secrets.MY_TOKEN }}"

      - name: Use step output
        id: version
```

```
run: echo "tag=v1.2.3" >> $GITHUB_OUTPUT

- name: Read steps context
  run: echo "Version is ${ steps.version.outputs.tag }"
```

Explanation of each context

github

`github` gives metadata about the repository and the event that triggered the workflow, such as `github.repository`, `github.ref`, `github.actor`, and `github.event`. It is one of the most important contexts for interview questions because it controls branch-based logic and PR automation. ^[2] ^[1]

Example:

```
run: echo "Branch is ${ github.ref }"
```

env

`env` is used for environment variables defined at workflow, job, or step level. It is best for non-sensitive configuration values like environment names, feature flags, or app modes. ^[1]

Example:

```
env:
  APP_ENV: staging
run: echo "Deploying to ${ env.APP_ENV }"
```

runner

`runner` contains information about the machine running the job, such as `runner.os`, `runner.arch`, and temporary paths. This is useful when you want OS-specific logic in your pipeline. ^[3] ^[1]

Example:

```
run: echo "Running on ${ runner.os }"
```

secrets

`secrets` stores encrypted sensitive values like API keys, tokens, and passwords. You should use it only for sensitive data and avoid printing it in logs. ^[3] ^[1]

Example:

```
env:
  TOKEN: ${ secrets.MY_TOKEN }
run: curl -H "Authorization: Bearer $TOKEN" https://api.example.com
```

steps

steps lets you read outputs from earlier steps in the same job. This is useful when one step computes a value and another step needs to reuse it. ^[2] ^[1]

Example:

```
- name: Set image tag
  id: image
  run: echo "tag=1.0.0" >> $GITHUB_OUTPUT

- name: Use image tag
  run: echo "Tag is ${ steps.image.outputs.tag }"
```

needs

needs is used when one job depends on another job and needs its outputs or status. It is very common in build-test-deploy pipelines. ^[4] ^[3]

Example:

```
jobs:
  build:
    runs-on: ubuntu-latest
    outputs:
      image_tag: ${ steps.build.outputs.tag }
    steps:
      - id: build
        run: echo "tag=v2.0.0" >> $GITHUB_OUTPUT

  deploy:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying ${ needs.build.outputs.image_tag }"
```

matrix

matrix is used in matrix builds, where the same job runs with multiple parameter combinations. It is ideal for testing across Node versions, Python versions, or operating systems. ^[5] ^[1]

Example:

```
strategy:
  matrix:
    node-version: [18, 20]

steps:
  - run: echo "Testing Node ${ matrix.node-version }"
```

Common interview example

A very typical interview example is branch-based deployment:

```
name: Deploy

on: push

jobs:
  deploy:
    if: ${ github.ref == 'refs/heads/main' }
    runs-on: ubuntu-latest
    steps:
      - run: echo "Deploying from main branch"
```

This works because `github.ref` tells GitHub Actions which branch or tag triggered the workflow, and the `if` condition decides whether the job should run. ^[1]

How to explain in interview

You can say:

“GitHub Actions contexts are built-in objects like `github`, `env`, `secrets`, `runner`, `steps`, `needs`, and `matrix`. They let workflows access runtime data dynamically, so instead of hardcoding values, we can make CI/CD pipelines conditional, reusable, and secure.” ^[2] ^[1]

One important caution

Do not dump secrets or the full context to logs in a real workflow unless you are debugging carefully, because the context may contain sensitive information. ^[2] ^[1]

If you need to inspect a context, use `toJson()` only in safe environments and with caution. ^[2]

Would you like a **single full GitHub Actions YAML** that combines `github`, `env`, `secrets`, `matrix`, `steps`, and `needs` in one interview-ready example?

✴✴

1. <https://docs.github.com/en/actions/concepts/workflows-and-actions/contexts>

2. <https://github.com/orgs/community/discussions/134460>

3. <https://docs.github.com/en/actions/reference/workflows-and-actions/contexts>

4. <https://github.com/orgs/community/discussions/27048>

5. https://www.youtube.com/watch?v=KY3vI_RjVsw
6. <https://github.blog/changelog/2025-09-18-actions-yaml-anchors-and-non-public-workflow-templates/>
7. <https://docs.github.com/actions/using-workflows/workflow-syntax-for-github-actions>
8. <https://stackoverflow.com/questions/70104600/complete-list-of-github-actions-contexts>
9. <https://dev.to/donnyc1/automating-your-workflow-with-github-actions-3o09>